



Module 7: Building AI Native Applications with Streamlit and Gradio

Demo 1: Streaming AI Responses Live in a Streamlit UI

Objective

By the end of this hands-on, you will be able to:

- Build a Streamlit-based AI chat application.
- Create a FastAPI backend that streams LLM responses using Server-Sent Events.
- Connect Streamlit to a FastAPI streaming endpoint.
- Send complete chat history from the frontend to the backend.
- Stream AI responses live from the Groq LLM API.
- Display token-by-token responses in a chat-style interface.
- Maintain conversation history using Streamlit session state.
- Securely manage API keys using environment variables.
- Handle backend request errors gracefully in a user-facing AI application.
- Understand how modern AI assistants deliver real-time interactive responses.

Introduction



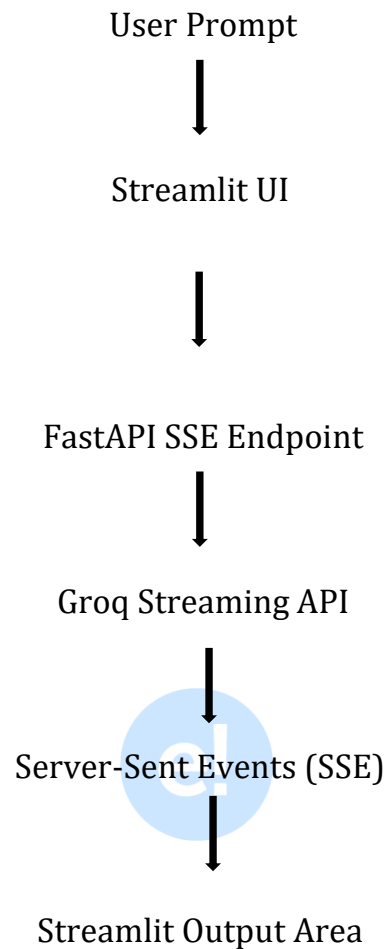
Many AI applications do not wait for the entire response before showing output. Instead, they stream the response gradually, word by word or token by token. This creates a smoother and faster experience for users.

For example, when a user asks a question in an AI chat interface, the answer appears progressively instead of loading all at once at the end.

In this hands-on, you will build a Streamlit chat application that sends the complete conversation history to a FastAPI backend. The FastAPI backend will call the Groq LLM API, stream the generated response using Server-Sent Events, and send each response chunk back to Streamlit. The Streamlit interface will then display the answer live as it is generated.

Application Architecture

Before writing code, understand the flow of the application.



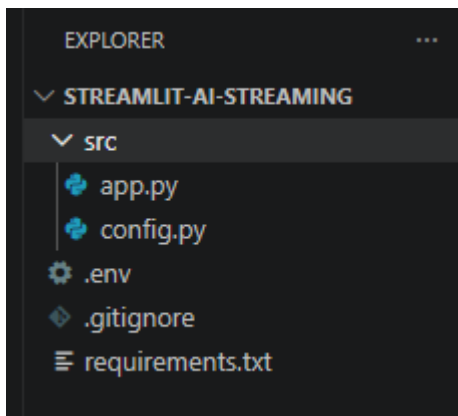
Request Flow

- The user enters a prompt in the Streamlit chat interface.
- Streamlit sends the complete chat history to the FastAPI /stream endpoint.
- FastAPI validates the request and sends it to the Groq LLM API with streaming enabled.
- Groq streams the response back to FastAPI, and FastAPI forwards it to Streamlit using SSE.
- Streamlit displays the response live and stores the final answer in chat history.

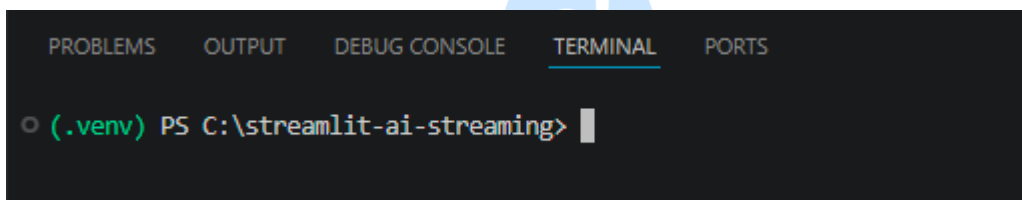
Project Setup

Before building the Streamlit application, prepare a new project environment for this hands-on.

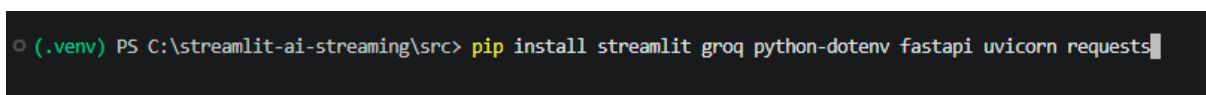
1. Create a new project folder and the required files for the Streamlit-based AI application.



2. Create and activate the virtual environment.



3. Install the required packages and save the dependencies.



4. Configure environment variables.



5. Protect sensitive files.

```
.gitignore X
.gitignore
1 # Environment Variables
2 .env
3
4 # Python Cache
5 __pycache__/_
6
7 # VS Code
8 .vscode/
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(.venv) PS C:\streamlit-ai-streaming> git init
Initialized empty Git repository in C:/streamlit-ai-streaming/.git/
(.venv) PS C:\streamlit-ai-streaming> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        requirements.txt
        src/

nothing added to commit but untracked files present (use "git add" to track)
(.venv) PS C:\streamlit-ai-streaming> 
```

6. Create the configuration module.

```
config.py U X
src > config.py > ...
1 from dotenv import load_dotenv
2 import os
3 load_dotenv()
4 GROQ_API_KEY = os.getenv("GROQ_API_KEY")
5 MODEL_NAME = os.getenv("MODEL_NAME")
6 ENVIRONMENT = os.getenv("ENVIRONMENT")
7
```

Creating the FastAPI SSE Backend

Before building the Streamlit interface, create a FastAPI backend that streams LLM responses using Server-Sent Events.

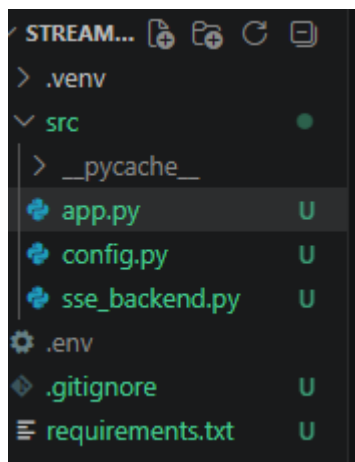
This backend will receive a prompt from Streamlit, send it to the Groq LLM, and stream the generated response back chunk by chunk.

Step 1 – Create the Backend File

Inside the project folder, create a new file named:

sse_backend.py

Your project structure should now look like this:



Step 2 – Add the FastAPI SSE Code

Open sse_backend.py and add the following code:

```

sse_backend.py U X
src > sse_backend.py > ...
1  import json
2  from typing import List
3  from fastapi import FastAPI
4  from fastapi.responses import StreamingResponse
5  from pydantic import BaseModel
6  from groq import Groq
7  from config import GROQ_API_KEY, MODEL_NAME
8  app = FastAPI(
9      title="FastAPI SSE Streaming Backend",
10     description="Streams Groq LLM responses to a Streamlit frontend using true SSE",
11     version="2.0.0"
12 )
13 client = Groq(api_key=GROQ_API_KEY)
14 class ChatMessage(BaseModel):
15     role: str
16     content: str
17 class ChatRequest(BaseModel):
18     messages: List[ChatMessage]
19 @app.get("/")
20 def home():
21     return {"message": "FastAPI SSE backend is running."}
22 @app.post("/stream")
23 def stream_response(request: ChatRequest):
24     def generate():
25         stream = client.chat.completions.create(
26             model=MODEL_NAME,
27             messages=[m.dict() for m in request.messages],
28             temperature=0.2,
29             max_tokens=500,
30             stream=True,
31         )
32         for chunk in stream:
33             content = chunk.choices[0].delta.content
34             if content:
35                 payload = json.dumps({"content": content})
36                 yield f"data: {payload}\n\n"
37         yield "data: [DONE]\n\n"
38     return StreamingResponse(
39         generate(),

```

```

40     media_type="text/event-stream",
41     headers={
42         "Cache-Control": "no-cache",
43         "X-Accel-Buffering": "no",
44     },
45 )
46 @app.post("/generate")
47 def generate_response(request: ChatRequest):
48     response = client.chat.completions.create(
49         model=MODEL_NAME,
50         messages=[m.dict() for m in request.messages],
51         temperature=0.2,
52         max_tokens=500,
53     )
54     return {"response": response.choices[0].message.content}
55
56
57
58
59
60

```

Code Explanation:

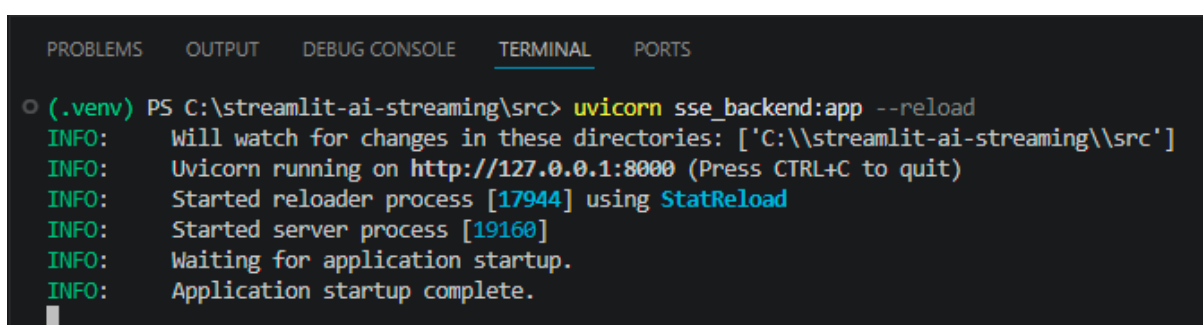
- **import json** imports Python's JSON module, which is used to convert streamed text chunks into JSON format before sending them to the frontend.
- **from typing import List** imports List, which is used to define a list of chat messages in the request body.
- **from fastapi import FastAPI** imports the FastAPI framework used to create the backend API application.
- **from fastapi.responses import StreamingResponse** imports the StreamingResponse class, which allows the backend to stream data gradually instead of returning the complete response at once.
- **from pydantic import BaseModel** imports BaseModel, which is used to create structured request models and validate incoming data.
- **from groq import Groq** imports the Groq client used to communicate with the Groq LLM API.
- **from config import GROQ_API_KEY, MODEL_NAME** imports the API key and language model name from the configuration file.
- **app = FastAPI(...)** creates the FastAPI application and adds metadata such as the title, description, and version.
- **client = Groq(api_key=GROQ_API_KEY)** creates a Groq client using the API key loaded from the configuration file.
- **class ChatMessage(BaseModel):** defines the structure of a single chat message sent from the Streamlit frontend.
- **role: str** stores the role of the message, such as user or assistant.
- **content: str** stores the actual message text.
- **class ChatRequest(BaseModel):** defines the structure of the complete request sent to the backend.
- **messages: List[ChatMessage]** specifies that the request must contain a list of chat messages, allowing the backend to receive the full conversation history.
- **@app.get("/")** defines a root endpoint used to check whether the FastAPI backend is running.

- **def home():** defines the function that runs when the root URL is opened.
- **return {"message": "FastAPI SSE backend is running."}** returns a simple JSON response confirming that the backend is active.
- **@app.post("/stream")** creates a POST endpoint that streams the AI response using Server-Sent Events.
- **def stream_response(request: ChatRequest):** accepts the validated chat request sent by the Streamlit frontend.
- **def generate():** defines a generator function that produces response chunks one at a time.
- **client.chat.completions.create(...)** sends the conversation messages to the Groq LLM API.
- **model=MODEL_NAME** specifies which Groq model should generate the response.
- **messages=[m.dict() for m in request.messages]** converts the Pydantic message objects into dictionary format required by the Groq API.
- **temperature=0.2** controls response randomness and keeps the output more focused and consistent.
- **max_tokens=500** limits the maximum length of the generated response.
- **stream=True** enables streaming so the model returns the response incrementally instead of waiting for the full answer.
- **for chunk in stream:** loops through each streamed response chunk returned by Groq.
- **content = chunk.choices[0].delta.content** extracts the newly generated text from the current streamed chunk.
- **if content:** checks whether the current chunk contains visible text before sending it to the frontend.
- **payload = json.dumps({"content": content})** converts the streamed text into JSON format so spacing, symbols, and new lines are preserved correctly.
- **yield f"data: {payload}\n\n"** sends each chunk in proper Server-Sent Events format.

- **yield "data: [DONE]\n\n"** sends a completion signal so the Streamlit frontend knows the response has finished streaming.
- **return StreamingResponse(...)** returns the generator as a live streaming response.
- **generate()** is passed to StreamingResponse so each generated chunk can be sent as soon as it becomes available.
- **media_type="text/event-stream"** tells the client that the response is being sent using Server-Sent Events.
- **headers={...}** adds response headers that help the stream behave correctly.
- **"Cache-Control": "no-cache"** prevents the streamed response from being cached.
- **"X-Accel-Buffering": "no"** helps disable buffering when the application is deployed behind servers such as Nginx.
- **@app.post("/generate")** creates a standard non-streaming endpoint that returns the complete AI response at once.
- **def generate_response(request: ChatRequest):** accepts the same chat request format for a non-streaming response.
- **response = client.chat.completions.create(...)** sends the conversation messages to Groq and waits until the full response is generated.
- **return {"response": response.choices[0].message.content}** extracts the complete generated response and returns it as a JSON object.

Step 3 – Run the FastAPI SSE Backend

Open a terminal and run:



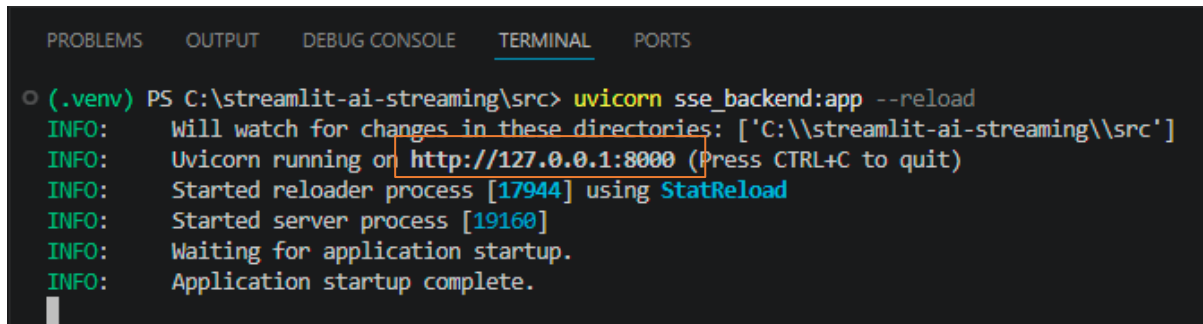
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
○ (.venv) PS C:\streamlit-ai-streaming\src> uvicorn sse_backend:app --reload
INFO:      Will watch for changes in these directories: ['C:\streamlit-ai-streaming\src']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [17944] using StatReload
INFO:      Started server process [19160]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
  
```

Step 4 - Test the Backend in the Browser

Open the following URL in your browser:

<http://127.0.0.1:8000/>



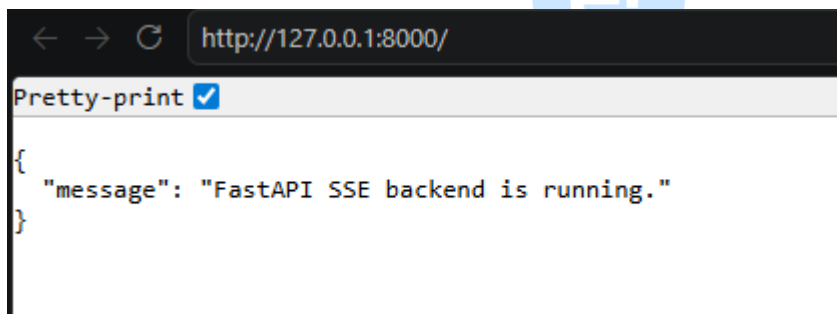
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\streamlit-ai-streaming\src> uvicorn sse_backend:app --reload
INFO: Will watch for changes in these directories: ['C:\streamlit-ai-streaming\src']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [17944] using StatReload
INFO: Started server process [19160]
INFO: Waiting for application startup.
INFO: Application startup complete.
  
```

Expected response:

```

{
  "message": "FastAPI SSE backend is running."
}
  
```



```

http://127.0.0.1:8000/
Pretty-print [checked]
{
  "message": "FastAPI SSE backend is running."
}
  
```

Building the Streamlit Streaming Application

Now create the Streamlit frontend that sends user prompts to the FastAPI streaming backend and displays the response live as it arrives. The Streamlit app will not call Groq directly. Instead, it will call the /stream endpoint created in the FastAPI backend.

Step 1 – Build the Streamlit Application

Open `app.py` and add

```

src > app.py U X
1  import json
2  import requests
3  import streamlit as st
4  BACKEND_STREAM_URL = "http://127.0.0.1:8000/stream"
5  st.set_page_config(page_title="Live AI Chat Stream", layout="centered")
6  st.title("Live AI Chat Stream")
7  st.write("Ask a question and watch the response stream from a FastAPI backend using SSE.")
8  if "messages" not in st.session_state:
9      st.session_state.messages = []
10 for message in st.session_state.messages:
11     with st.chat_message(message["role"]):
12         st.markdown(message["content"])
13 prompt = st.chat_input("Ask something about Agentic AI...")
14 if prompt:
15     st.session_state.messages.append({"role": "user", "content": prompt})
16     with st.chat_message("user"):
17         st.markdown(prompt)
18     with st.chat_message("assistant"):
19         response_box = st.empty()
20         full_response = ""
21         try:
22             response = requests.post(
23                 BACKEND_STREAM_URL,
24                 json={"messages": st.session_state.messages},
25                 stream=True,
26                 timeout=60,
27             )
28             response.raise_for_status()
29             for line in response.iter_lines(decode_unicode=True):
30                 if not line:
31                     continue
32                 if line.startswith("data: "):
33                     data_str = line[len("data: "):]
34
35                     if data_str == "[DONE]":
36                         break
37                     try:
38                         chunk = json.loads(data_str)
39                         token = chunk.get("content", "")
40
41                         full_response += token
42                         response_box.markdown(full_response + "▮")
43                     except json.JSONDecodeError:
44                         continue
45                 response_box.markdown(full_response)
46                 st.session_state.messages.append(
47                     {"role": "assistant", "content": full_response}
48                 )
49             except requests.exceptions.RequestException as error:
50                 st.error(f"Backend request failed: {error}")
51             except Exception as error:
52                 st.error(f"An error occurred: {error}")
53

```

Understanding the Streamlit Code

- **import json** imports Python's JSON module, which is used to read the JSON-formatted chunks received from the FastAPI SSE backend.
- **import requests** imports the requests library, which is used to send HTTP requests from the Streamlit frontend to the FastAPI backend.
- **import streamlit as st** imports Streamlit and gives it the alias st, which is used to build the web-based chat interface.
- **BACKEND_STREAM_URL = "<http://127.0.0.1:8000/stream>"** stores the URL of the FastAPI streaming endpoint.
- **st.set_page_config(page_title="Live AI Chat Stream", layout="centered")** configures the Streamlit page title and sets the layout to centred.
- **st.title("Live AI Chat Stream")** displays the main title of the Streamlit application.
- **st.write("Ask a question and watch the response stream from a FastAPI backend using SSE.")** displays a short description explaining what the application does.
- **if "messages" not in st.session_state:** checks whether the chat history already exists in Streamlit session state.
- **st.session_state.messages = []** creates an empty list to store the conversation history if it does not already exist.
- **for message in st.session_state.messages:** loops through all previously stored chat messages.
- **with st.chat_message(message["role"]):** displays each message using the correct chat role, such as user or assistant.
- **st.markdown(message["content"])** renders the message content in the chat interface.

- **prompt = st.chat_input("Ask something about Agentic AI...")** creates a chat input box where the user can type a question.
- **if prompt:** checks whether the user has submitted a new message.
- **st.session_state.messages.append({"role": "user", "content": prompt})** stores the user's message in the conversation history.
- **with st.chat_message("user"):** creates a user message block in the chat interface.
- **st.markdown(prompt)** displays the user's submitted prompt immediately.
- **with st.chat_message("assistant"):** creates an assistant message block where the streamed response will appear.
- **response_box = st.empty()** creates an empty placeholder that can be updated repeatedly while the response is streaming.
- **full_response = ""** creates an empty string to store the complete assistant response as chunks arrive.
- **try:** starts a block of code where backend communication and streaming are attempted safely.
- **response = requests.post(...)** sends a POST request from Streamlit to the FastAPI /stream endpoint.
- **BACKEND_STREAM_URL** tells the request where the FastAPI streaming endpoint is running.
- **json={"messages": st.session_state.messages}** sends the complete chat history to the backend in JSON format.
- **stream=True** keeps the HTTP connection open so the frontend can receive streamed chunks gradually.
- **timeout=60** prevents the request from waiting forever if the backend does not respond.

- **response.raise_for_status()** checks whether the backend returned a successful HTTP response and raises an error for failed requests.
- **for line in response.iter_lines(decode_unicode=True):** reads the streamed SSE response line by line.
- **if not line:** skips empty lines in the SSE stream.
- **continue** moves to the next line when the current line is empty or invalid.
- **if line.startswith("data: "):** checks whether the line is a valid SSE data message.
- **data_str = line[len("data: "):]** removes the data: prefix and keeps only the actual streamed content.
- **if data_str == "[DONE]":** checks whether the backend has sent the completion signal.
- **break** stops reading the stream once the response is complete.
- **try:** starts a nested block to safely parse each streamed JSON chunk.
- **chunk = json.loads(data_str)** converts the streamed JSON string into a Python dictionary.
- **token = chunk.get("content", "")** extracts the generated text from the current response chunk.
- **full_response += token** adds the new token to the complete assistant response.
- **response_box.markdown(full_response + "▮ ")** updates the Streamlit chat window with the latest response and shows a cursor effect while streaming.
- **except json.JSONDecodeError:** catches invalid JSON chunks if any malformed stream data is received.

- **continue** skips the invalid chunk and continues reading the remaining stream.
- **response_box.markdown(full_response)** displays the final assistant response after streaming is complete, without the cursor symbol.
- **st.session_state.messages.append({"role": "assistant", "content": full_response})** stores the final assistant response in chat history.
- **except requests.exceptions.RequestException as error:** catches backend request errors, such as connection failure, timeout, or HTTP error.
- **st.error(f"Backend request failed: {error}")** displays a user-friendly error message if the FastAPI backend request fails.
- **except Exception as error:** catches any other unexpected error in the application.
- **st.error(f"An error occurred: {error}")** displays a general error message in the Streamlit interface.



What This Step Does

This step converts the Streamlit app into a frontend client for the FastAPI SSE backend. Instead of calling Groq directly, Streamlit sends the complete chat history to FastAPI. FastAPI handles the LLM streaming process and sends response chunks back to Streamlit using Server-Sent Events. Streamlit then displays the response live and stores the final answer in the chat history.

Step 2 – Run the Streamlit Application

Run:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● (.venv) PS C:\streamlit-ai-streaming> cd src
○ (.venv) PS C:\streamlit-ai-streaming\src> streamlit run app.py

```

Expected Output

Live AI Chat Stream

Ask a question and watch the AI response stream live.

Ask something about Agentic AI...



Step 3 – Test the Application



In the prompt box, enter **“Explain Agentic AI in simple terms.”**

Explain Agentic AI in simple terms



Click **submit icon**

Expected behavior :

- The user message appears in the chat window.
- The assistant response starts appearing quickly.
- Text appears gradually instead of all at once.
- The final response remains visible in the chat history.
- The conversation can continue with another prompt.

Ask a question and watch the AI response stream live.



Explain Agentic AI in simple terms



Agentic AI refers to artificial intelligence (AI) systems that can act independently, making decisions and taking actions on their own, without being explicitly programmed for every situation.

Think of it like this: Imagine you have a personal assistant who can not only follow your instructions but also anticipate what you might need and take care of it before you ask. This assistant can learn from experience, adapt to new situations, and make decisions based on its own understanding of what's best.

Agentic AI has three key characteristics:

1. **Autonomy:** It can operate independently, making decisions without human intervention.
2. **Agency:** It can take actions that affect the environment or other systems.
3. **Intentionality:** It has goals and can work towards achieving them.

In simple terms, Agentic AI is like a smart, self-directed robot that can think, learn, and act on its own, making it a powerful tool for various applications, from healthcare to finance to transportation.

Ask something about Agentic AI...



What to Observe



- The app now behaves like a chat interface
- The UI does not wait for the full answer before showing output.
- The response appears progressively because stream=True is enabled.
- The conversation history remains visible because messages are stored in st.session_state.

Conclusion

In this hands-on, you built a real-time AI chat application using Streamlit as the frontend and FastAPI as the streaming backend. You created a chat interface, accepted user prompts, sent the complete conversation history to a backend endpoint, streamed LLM-generated responses, and displayed the output progressively as it arrived.

You also used Streamlit session state to preserve conversation history, making the application behave more like a real AI assistant instead of a one-time prompt-response tool. By separating the frontend from the backend, you followed a cleaner and more practical architecture for user-facing AI applications.

This workflow forms the foundation for building responsive AI chatbots, copilots, research assistants, knowledge assistants, and other interactive Agentic AI applications where speed, usability, and clean system design are important.

